

A REPORT
ON
MUP TOOL MODERNIZATION

Manageable Update Package Framework Optimization

BY

Name of the Student	ID No.	Discipline
Shreejeet Mishra	2022A7PS0036H	Computer Science

Prepared in partial fulfillment of the
Practice School - II

AT

Micron Technology Inc.

Hyderabad, India

A Practice School - II Station of

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

May, 2026

Practice School Division

Station: Micron Technology Inc.

Centre: Hyderabad

Duration: 6 Months

Date of Start: January 2026

Date of Submission: May 2026

Title of the Project: MUP Tool Modernization: Framework Optimization for Enterprise SSD Firmware Updates

ID No., Name and Discipline of the student:

2022A7PS0036H — Shreejeet Mishra — Computer Science and Engineering

Team:

cSSD Software Team, Micron Technology

Name of the PS faculty member:

Dr. Pawan Sharma

Key words:

Firmware Update, Win32 API, GUI Modernization, Validation Framework, Automated Testing, Enterprise Software, Cost Optimization, Performance Engineering

Project area:

Systems Software Engineering, Firmware Management, Enterprise Tooling

Abstract:

This project addresses the modernization of Micron's Manageable Update Package (MUP) tool, a critical enterprise software used by OEM partners like Dell for field firmware updates of SSDs. The existing Qt-based implementation imposed significant licensing costs (4.25L per developer annually), memory overhead (80MB RAM), and lacked automation. The modernized solution utilizes Win32 API for GUI rendering,

achieving 57% executable size reduction, 58% RAM usage reduction, and eliminating licensing costs entirely. Additionally, a comprehensive 206-test validation framework was developed using Python and pywinauto, incorporating static, functional, GUI, and integration tests. The implementation demonstrates that legacy GUI frameworks can be successfully replaced with native OS APIs while maintaining functional parity, significantly reducing operational costs and resource consumption in enterprise environments.

Acknowledgments

I would like to express my sincere gratitude to **Micron Technology Inc.** for providing me with the opportunity to work on this impactful project as part of my Practice School-II program. This experience has been instrumental in developing my understanding of enterprise software engineering, performance optimization, and industry-standard development practices.

I am deeply thankful to the **cSSD Software Team** at Micron Hyderabad for their continuous guidance, technical mentorship, and collaborative support throughout this project. Their expertise in firmware management systems and commitment to code quality has significantly shaped my professional growth.

Special thanks to my team members and colleagues who provided valuable insights during design discussions, code reviews, and validation phases. Their feedback was crucial in achieving the project's technical and business objectives.

I extend my gratitude to **BITS Pilani** and the **Practice School Division** for structuring a program that bridges academic learning with real-world industry challenges. The exposure to production-grade software development has been invaluable.

Finally, I acknowledge the open-source community for tools and libraries that supported the development and testing phases of this project, including Python, pywinauto, and the rich terminal UI library.

Shreejeet Mishra

2022A7PS0036H

Abstract

The Manageable Update Package (MUP) tool is a critical enterprise software solution developed by Micron Technology for enabling OEM partners to perform offline firmware updates on SSDs deployed in production environments. The tool is delivered as a standalone executable package for every firmware release, requiring validation in OEM environments such as Dell's Dominion platform.

The existing implementation, built using the Qt GUI framework, presented significant challenges: (1) high licensing costs of approximately 4.25 lakh per developer annually, (2) large executable size exceeding 30MB, (3) excessive RAM consumption of up to 80MB, (4) dependency on heavyweight rendering libraries unsuitable for the tool's simple UI requirements, and (5) manual, labor-intensive build and packaging processes.

This project successfully modernized the MUP tool by replacing Qt with native Win32 API and GDI+ for GUI rendering, eliminating external framework dependencies while maintaining complete visual and functional parity with the original interface. The re-engineered architecture follows a layered design with clear separation between backend firmware operations, platform abstraction, and UI layers.

Key technical achievements include: (1) 57% reduction in executable size (from 30MB+ to 13MB), (2) 58% reduction in RAM usage (from 80MB to 34MB), (3) complete elimination of annual licensing costs, (4) full automation of the build pipeline using Jenkins, and (5) development of a comprehensive 206-test validation framework covering static analysis, functional testing, GUI automation, and end-to-end integration scenarios.

The validation framework, implemented in Python using pywinauto for GUI automation, ensures reliability across different DPI settings and screen resolutions through dynamic coordinate-based interaction. The framework includes both standard output and a beautiful terminal UI variant using the rich library, providing real-time progress tracking and color-coded test results.

This modernization demonstrates that enterprise-grade software can achieve dramatic cost and resource savings through careful framework selection and architecture optimization without sacrificing user experience or functionality. The project resulted in mea-

surable productivity improvements of 16 story points in the MS2 sprint, validating the business value of technical debt reduction initiatives.

Keywords: Firmware Update Tools, GUI Framework Migration, Win32 API, Performance Optimization, Cost Reduction, Automated Testing, pywinauto, Jenkins CI/CD, Enterprise Software Engineering, Technical Debt Reduction

Contents

Acknowledgments	4
Abstract	5
1 Introduction	1
1.1 Background	1
1.2 Problem Context	1
1.3 Project Scope	2
1.4 Objectives	3
1.5 Significance	3
1.6 Report Organization	4
2 Existing System Analysis	5
2.1 MUP Tool Architecture (Qt-based)	5
2.1.1 Backend Components	5
2.1.2 Frontend Components	5
2.1.3 Integration Layer	6
2.2 MUP Workflow Pipeline	6
2.2.1 Tool Generation Workflow	6
2.2.2 OEM Validation Workflow	7
2.3 Operational Modes	7
2.3.1 Attended Mode	7
2.3.2 Unattended Mode	8
2.4 Identified Limitations	8
2.4.1 Financial Impact	8
2.4.2 Performance Impact	9
2.4.3 Development Impact	9
2.4.4 Deployment Impact	9
2.5 Technical Debt Assessment	9
3 Problem Statement and Justification	10
3.1 Core Problem Definition	10

3.2	Quantified Problem Evidence	10
3.2.1	Licensing Cost Analysis	10
3.2.2	Binary Size Analysis	11
3.2.3	Memory Footprint Analysis	11
3.3	Framework Over-Engineering Analysis	11
3.4	Build Process Inefficiencies	12
3.5	Automation Gap Analysis	12
3.6	OEM Compatibility Requirements	13
3.7	Business Impact Assessment	13
3.8	Justification for Modernization	14
4	Proposed Solution and Framework Evaluation	14
4.1	Solution Requirements	14
4.1.1	Functional Requirements	14
4.1.2	Non-Functional Requirements	15
4.2	Framework Evaluation Methodology	15
4.2.1	Evaluation Criteria	15
4.3	Candidate Framework Analysis	16
4.3.1	Option 1: Qt (Status Quo)	16
4.3.2	Option 2: Tauri + Rust	16
4.3.3	Option 3: Win32 API + GDI/GDI+	17
4.4	Framework Comparison Matrix	18
4.5	Recommended Solution Architecture	18
4.5.1	Backend Layer (Unchanged)	19
4.5.2	Platform Abstraction Layer	19
4.5.3	GUI Layer (New - Win32/GDI+)	19
4.5.4	Automation Layer (New)	19
4.6	Migration Strategy	20
4.7	Risk Assessment and Mitigation	21
4.8	Expected Outcomes	21
5	Future Enhancements	22
5.1	Jira MCP Integration with Jenkins for End-to-End Automation	22

5.1.1	Current Workflow Limitations	22
5.1.2	Proposed Integration Architecture	22
5.1.3	MCP Protocol Benefits	24
5.1.4	Implementation Roadmap	24
5.2	Additional Future Enhancements	24
5.2.1	Enhanced Security Features	25
5.2.2	Advanced Testing Capabilities	25
5.2.3	Monitoring and Observability	25
5.2.4	User Experience Improvements	25
5.3	Expected Impact of Future Enhancements	26
6	Conclusion	26
6.1	Achievement Summary	26
6.2	Technical Contributions	27
6.3	Lessons Learned	27
6.4	Organizational Impact	28
6.5	Industry Relevance	28
6.6	Final Remarks	29
	Glossary	29

List of Tables

1	Qt Licensing Cost Breakdown	10
2	Executable Size Breakdown (Qt-based)	11
3	Runtime Memory Usage (Qt-based)	11
4	Qt Feature Utilization	12
5	Automation Maturity Assessment	13
6	Framework Evaluation Criteria	15
7	Comprehensive Framework Comparison	18
8	Risk Mitigation Matrix	21

1 Introduction

1.1 Background

Modern data centers and enterprise computing environments rely heavily on Solid State Drives (SSDs) for storage infrastructure. Unlike consumer-grade storage devices, enterprise SSDs require periodic firmware updates to address security vulnerabilities, improve performance characteristics, enhance reliability, and add new features demanded by evolving workload requirements.

Micron Technology, as a leading manufacturer of enterprise storage solutions, provides firmware update tools to Original Equipment Manufacturers (OEMs) like Dell, HP, Lenovo, and others. These OEMs integrate Micron's SSDs into their server products and require the ability to update firmware in field-deployed systems without requiring online connectivity or direct manufacturer support.

The Manageable Update Package (MUP) tool serves this critical function. For every production firmware release, Micron packages the firmware binary along with a user interface, update logic, and validation checks into a standalone executable. This package is delivered to OEM partners, who then deploy it through their management platforms (such as Dell's Dominion system) to update SSDs across their enterprise customer base.

1.2 Problem Context

The existing MUP tool implementation, developed several years ago, utilized the Qt framework for its graphical user interface. Qt is a cross-platform application development framework widely used for desktop applications with complex UI requirements. However, for the MUP tool's relatively simple interface (consisting of license acceptance dialogs, drive selection dropdowns, progress bars, and status messages), Qt represented significant over-engineering.

This technical choice created several operational and financial burdens:

- **High Licensing Costs:** Qt's commercial license costs approximately \$4,660 per developer per year (4.25 lakh), which accumulates significantly across multi-developer

teams over multiple years.

- **Large Binary Size:** The Qt framework requires bundling substantial runtime libraries (QtCore, QtGui, QtWidgets, platform plugins), resulting in executables exceeding 30MB even before including firmware payloads.
- **Excessive Memory Consumption:** Runtime memory usage peaked at 80MB due to Qt's rendering engine, scene graph, and widget hierarchy management — far beyond what simple UI dialogs require.
- **Dependency Management Complexity:** Ensuring Qt DLLs, platform plugins, and specific versions were correctly packaged introduced deployment complexity and potential compatibility issues.
- **Manual Build Processes:** The tool build, packaging, and validation workflow was largely manual, requiring developer intervention for each firmware release and increasing time-to-delivery.

1.3 Project Scope

This project aimed to comprehensively modernize the MUP tool by addressing all identified inefficiencies while preserving complete backward compatibility with OEM validation systems. The scope encompassed:

1. **GUI Framework Migration:** Replace Qt with native Win32 API and GDI+ for rendering, eliminating licensing costs and external dependencies.
2. **Architecture Redesign:** Implement a clean layered architecture separating firmware logic, platform abstraction, and UI concerns.
3. **Performance Optimization:** Reduce executable size and runtime memory footprint through efficient coding practices and removal of unnecessary abstractions.
4. **Build Automation:** Develop Jenkins-based CI/CD pipelines to automate tool generation, packaging, and artifact delivery.

5. **Comprehensive Validation Framework:** Create an automated test suite covering static analysis (72 tests), functional firmware operations (1 test), GUI interaction scenarios (5 tests), and end-to-end integration workflows (1 test), totaling 79+ validation checks.
6. **Documentation and Knowledge Transfer:** Document architectural decisions, API interfaces, testing methodologies, and operational procedures for long-term maintainability.

1.4 Objectives

The primary objectives of this modernization effort were:

1. Eliminate annual Qt licensing expenses, achieving zero-cost GUI framework.
2. Reduce executable size by at least 40% while maintaining all functionality.
3. Reduce runtime memory usage by at least 40% for improved system resource efficiency.
4. Maintain 100% visual and functional parity with the existing Qt-based UI to avoid OEM re-validation costs.
5. Automate the end-to-end build and packaging workflow to reduce manual effort and human error.
6. Develop a robust, maintainable validation framework to ensure quality across all firmware releases.
7. Achieve measurable productivity improvements in firmware release cycles.

1.5 Significance

This project holds significance on multiple dimensions:

Cost Reduction: Eliminating Qt licensing fees saves substantial recurring costs, with ROI realized within the first year of deployment across the cSSD team.

Technical Debt Reduction: Removing heavyweight framework dependencies simplifies the codebase, reduces maintenance burden, and improves long-term software sustainability.

Resource Efficiency: Lower memory and storage footprints enable deployment in resource-constrained environments and reduce environmental impact through decreased computational overhead.

Scalability: Automated pipelines support increased firmware release cadence without proportional increases in engineering effort.

Knowledge Contribution: The project demonstrates a successful framework migration methodology applicable to other internal tools facing similar challenges, providing a template for technical modernization initiatives.

1.6 Report Organization

The remainder of this report is organized as follows:

- **Chapter 2** analyzes the existing MUP tool architecture, workflow, and limitations in detail.
- **Chapter 3** presents the problem statement with quantitative evidence of issues.
- **Chapter 4** describes the proposed solution, including framework evaluation and selection rationale.
- **Chapter 5** details the Win32-based architecture and design patterns employed.
- **Chapter 6** covers implementation aspects, including GUI development and backend integration.
- **Chapter 7** presents the comprehensive validation framework architecture and testing methodology.
- **Chapter 8** provides quantitative results demonstrating achieved improvements.
- **Chapter 9** discusses lessons learned, challenges overcome, and best practices identified.

- **Chapter 10** outlines future enhancements and concludes the report.

2 Existing System Analysis

2.1 MUP Tool Architecture (Qt-based)

The original MUP tool followed a traditional desktop application architecture with Qt as the primary framework. The system consisted of several key components:

2.1.1 Backend Components

- **msecli.exe:** A command-line interface tool providing low-level firmware update operations. This executable communicated directly with SSD devices through operating system APIs and executed firmware flash operations.
- **mse-api.dll:** A dynamic link library encapsulating the Micron Storage Executive API, providing abstracted access to SSD management functions including device enumeration, health monitoring, and firmware update primitives.
- **Firmware Blob:** The actual firmware binary (packaged as a .zip or .bin file) containing the new firmware image to be written to SSD non-volatile memory.
- **mup.xml:** An XML configuration file containing metadata such as supported drive models, firmware version information, update prerequisites, and validation checksums.

2.1.2 Frontend Components

- **Qt Widgets:** The user interface was built using Qt's widget framework (QWidget, QPushButton, QProgressBar, QLabel, QComboBox) for creating windows, dialogs, buttons, and progress indicators.
- **Qt Core Libraries:** Event loop management (QCoreApplication), signal-slot mechanism for inter-component communication, and cross-platform abstractions.
- **Qt GUI Rendering:** The QPainter API and Qt's graphics scene graph for drawing UI elements with anti-aliasing, animations, and visual effects.

- **Qt Platform Plugins:** Platform-specific integration code (qwindows.dll on Windows) bridging Qt abstractions to native OS windowing systems.

2.1.3 Integration Layer

The Qt frontend and C/C++ backend were integrated through:

- Qt's signal-slot mechanism for asynchronous communication
- QProcess for spawning mseccli.exe and capturing output
- Direct DLL linking to mse-api.dll for API calls
- QXmlStreamReader for parsing mup.xml metadata

2.2 MUP Workflow Pipeline

The complete workflow from firmware release to customer deployment involved multiple manual steps:

2.2.1 Tool Generation Workflow

1. **Jira Request Initiation:** When a new firmware version was released, a Jira ticket (e.g., SDIRECTOR-15940) was filed requesting MUP tool generation for specific drive models.
2. **Firmware Image Integration:** The new firmware binary was manually copied into the project's resource directory.
3. **XML Configuration Update:** The mup.xml file was manually edited to reflect new firmware version numbers, checksums, and compatibility matrices.
4. **Qt Project Compilation:** Developers compiled the Qt project using Qt Creator or qmake/make build systems, ensuring Qt libraries matched the development environment version.
5. **Dependency Bundling:** Qt runtime DLLs (40MB+), platform plugins, and other dependencies were manually collected and packaged alongside the compiled executable.

6. **Installer Creation:** Tools like Enigma Virtual Box or NSIS were used to package everything into a single executable installer.
7. **Manual Validation:** Developers performed manual smoke testing on local systems to verify basic functionality.
8. **Delivery to OEM:** The final package was uploaded to an artifact repository or file sharing system for OEM partner access.

2.2.2 OEM Validation Workflow

OEM partners (particularly Dell) validated MUP tools using automated platforms:

1. **Dominion Platform Deployment:** Dell's Dominion system automated firmware update validation across multiple server configurations.
2. **Unattended Mode Testing:** MUP tools were executed with silent command-line flags (`/s /log="path"`) to verify headless operation for datacenter automation scenarios.
3. **Attended Mode Testing:** GUI validation ensured dialogs displayed correctly, user interactions functioned as expected, and error messages were clear.
4. **Result Classification:** Updates were classified as Pass (successful firmware application), Warning (conditional success with non-critical issues), or Fail (update aborted or device left in inconsistent state).

2.3 Operational Modes

The MUP tool supported two primary operational modes:

2.3.1 Attended Mode

In attended mode, the full graphical interface was presented:

- **License Agreement Screen:** Displayed EULA text with "I Accept" / "I Decline" buttons.

- **Drive Selection Screen:** Listed detected compatible SSDs with firmware version information.
- **Progress Screen:** Real-time progress bar with percentage completion and status messages.
- **Completion Screen:** Final result summary with success/failure indication and recommendations for next steps (e.g., system reboot requirements).

2.3.2 Unattended Mode

Unattended mode enabled silent, automated execution:

- Invoked via command-line: `Micron_4600_46000010.exe /s /log="C:\update.log"`
- No GUI displayed
- Firmware automatically applied to all compatible drives
- Results logged to specified file for batch processing systems
- Exit codes indicated success/failure for scripting integration

2.4 Identified Limitations

Through operational experience and metric collection, several critical limitations were identified:

2.4.1 Financial Impact

- Qt commercial licenses: \$4,660 per developer/year
- For a team of 5 developers: \$23,300/year (~21.25 lakh)
- Multi-year total cost of ownership exceeded 1 crore

2.4.2 Performance Impact

- Executable size: 30-35MB (excluding firmware payload)
- ZIP package size: 40-50MB (with firmware)
- RAM usage: 70-80MB at runtime
- Application startup time: 2-3 seconds

2.4.3 Development Impact

- Manual build process: 30-45 minutes per firmware release
- Dependency management overhead
- Qt version compatibility issues across developer workstations
- Limited automated testing capabilities
- High barrier to entry for new team members unfamiliar with Qt

2.4.4 Deployment Impact

- Large download sizes for OEM partners with limited bandwidth
- Increased storage requirements on OEM validation systems
- Potential DLL conflicts on end-user systems with other Qt-based applications

2.5 Technical Debt Assessment

Analysis revealed significant accumulated technical debt:

- Over-reliance on a heavyweight framework for simple UI requirements
- Lack of automated testing infrastructure
- Monolithic architecture making changes risky

- Inadequate documentation of build processes
- No version control for Qt dependencies

This assessment established a strong business case for modernization, which is explored in detail in the following chapters.

3 Problem Statement and Justification

3.1 Core Problem Definition

The central problem can be formulated as follows:

The existing Qt-based MUP tool implementation imposes unsustainable licensing costs, excessive resource consumption, and operational inefficiencies that hinder scalability of firmware release operations. A modernized solution is required that eliminates licensing expenses, reduces resource footprints by at least 40%, automates build processes, and maintains complete functional parity with existing OEM validation requirements.

3.2 Quantified Problem Evidence

3.2.1 Licensing Cost Analysis

According to Qt Company's official pricing (<https://www.qt.io/pricing>):

Table 1: Qt Licensing Cost Breakdown

Item	USD	INR (approx.)
Qt Commercial License (per developer/year)	\$4,660	4,25,000
Team of 5 developers (annual)	\$23,300	21,25,000
5-year total cost of ownership	\$116,500	1,06,25,000

This represents a significant recurring operational expense with no technical justification for the tool's simple UI requirements.

3.2.2 Binary Size Analysis

Table 2: Executable Size Breakdown (Qt-based)

Component	Size (MB)
Compiled application code	2.5
Qt5Core.dll	6.2
Qt5Gui.dll	7.8
Qt5Widgets.dll	5.6
Platform plugins	1.8
Image format plugins	3.2
Other Qt dependencies	4.1
Total (excluding firmware)	31.2

The Qt framework itself constituted over 90% of the executable size, with the actual application logic being negligible.

3.2.3 Memory Footprint Analysis

Task Manager analysis during typical operation:

Table 3: Runtime Memory Usage (Qt-based)

Memory Category	Usage (MB)
Working Set (RAM)	78.4
Private Bytes	71.2
Virtual Memory	124.6
Peak Working Set	82.1

For a tool that displays simple dialogs and progress bars, this memory consumption is excessive and unjustifiable.

3.3 Framework Over-Engineering Analysis

An examination of Qt framework features utilized versus available capabilities revealed:

Table 4: Qt Feature Utilization

Qt Capability	Available	Used in MUP
Advanced Widgets (QTreeView, QTableWidget)	Yes	No
Graphics View Framework	Yes	No
Animation Framework	Yes	No
WebEngine Integration	Yes	No
Networking	Yes	No
Database Connectivity	Yes	No
OpenGL Acceleration	Yes	No
Complex Layouts	Yes	No
Stylesheets	Yes	Minimal
Basic Widgets (Button, Label, Progress)	Yes	Yes

The MUP tool utilized less than 10% of Qt’s capabilities, representing significant framework overhead with no benefit.

3.4 Build Process Inefficiencies

Analysis of the manual build workflow identified several inefficiencies:

- **Human Error:** Manual XML editing led to configuration errors in 15% of releases
- **Time Consumption:** Average 35 minutes per build $\times$ 48 releases/year = 28 hours of developer time
- **Inconsistency:** Different developers used different Qt versions, causing occasional compatibility issues
- **Lack of Traceability:** No automated correlation between Jira tickets and generated artifacts

3.5 Automation Gap Analysis

The existing workflow lacked automation in critical areas:

Table 5: Automation Maturity Assessment

Process Area	Current State	Desired State
Build Compilation	Manual	Automated
Dependency Bundling	Manual	Automated
Configuration Generation	Manual	Automated
Packaging	Semi-automated	Automated
Validation Testing	Manual	Automated
Artifact Delivery	Manual	Automated
Jira Integration	None	Full

3.6 OEM Compatibility Requirements

Any modernization solution must maintain:

- Visual parity with existing GUI to avoid OEM re-validation costs
- Identical command-line interface for unattended mode
- Same exit codes and error reporting mechanisms
- Compatibility with Dell Dominion validation infrastructure
- Support for Windows 7, 8, 10, 11, and Server editions

Failure to meet these requirements would necessitate costly re-validation by multiple OEM partners, negating cost savings.

3.7 Business Impact Assessment

The cumulative business impact of these issues included:

- **Direct Costs:** 21.25L annual licensing for 5-developer team
- **Opportunity Costs:** Developer time spent on manual builds could be allocated to feature development

- **Scalability Limitations:** Manual processes constrained firmware release cadence
- **Competitive Disadvantage:** Longer time-to-market compared to competitors with automated pipelines
- **Technical Debt:** Accumulated maintenance burden reduced team velocity

3.8 Justification for Modernization

Given the quantified evidence of cost, performance, and operational inefficiencies, coupled with the availability of viable alternative technologies (Win32 API, Tauri), modernization of the MUP tool was strongly justified. The potential return on investment, even accounting for development effort, was projected to be realized within 6-12 months.

The following chapters detail the solution architecture and implementation approach developed to address these challenges.

4 Proposed Solution and Framework Evaluation

4.1 Solution Requirements

Based on the identified problems, the modernized solution needed to satisfy the following requirements:

4.1.1 Functional Requirements

1. Render identical user interface layouts as Qt version
2. Support both attended (GUI) and unattended (CLI) operational modes
3. Integrate with existing mseccli.exe and mse-api.dll backend components without modification
4. Parse mup.xml configuration files compatibly
5. Generate identical exit codes for automation system compatibility

6. Support Windows 7 through Windows 11 and Server editions
7. Handle high-DPI displays correctly (100%, 125%, 150%, 200% scaling)

4.1.2 Non-Functional Requirements

1. Eliminate all licensing costs (target: \$0 annual)
2. Reduce executable size by minimum 40% (target: <18MB)
3. Reduce runtime RAM usage by minimum 40% (target: <48MB)
4. Compile with standard MSVC toolchain without proprietary dependencies
5. Enable full build automation via CI/CD pipeline
6. Maintain or improve application startup time
7. Provide comprehensive automated test coverage

4.2 Framework Evaluation Methodology

Three candidate GUI frameworks were evaluated against the requirements:

4.2.1 Evaluation Criteria

Table 6: Framework Evaluation Criteria

Criterion	Description
Licensing Cost	Annual per-developer cost and total team cost
Binary Size	Executable and dependency footprint
Memory Usage	Runtime RAM consumption
Platform Support	Windows version compatibility
Development Effort	Learning curve and migration complexity
Maintainability	Long-term support and community health
Deployment Complexity	Installation and dependency management

4.3 Candidate Framework Analysis

4.3.1 Option 1: Qt (Status Quo)

Advantages:

- Already implemented, no migration effort required
- Mature framework with extensive documentation
- Cross-platform capabilities (though not needed for Windows-only tool)
- Rich widget library

Disadvantages:

- \$4,660/developer/year licensing cost
- 30MB+ executable size
- 80MB RAM usage
- Complex dependency management
- Over-engineered for simple UI requirements

Verdict: Does not meet cost or performance requirements.

4.3.2 Option 2: Tauri + Rust

Tauri is a modern framework for building desktop applications using web technologies (HTML/CSS/JavaScript) for the frontend and Rust for the backend.

Advantages:

- Zero licensing cost (MIT/Apache licensed)
- Modern, web-based UI development experience
- Potential for attractive, responsive interfaces

- Growing community and ecosystem
- Smaller executables compared to Electron

Disadvantages:

- Requires WebView2 runtime dependency (100MB+ on legacy systems)
- Learning curve for Rust programming language
- Backend integration complexity (Rust C/C++ FFI)
- Risk of UI behavior divergence from Qt version
- Potential OEM re-validation required due to visual differences
- Unproven in enterprise firmware tooling domain

Verdict: Introduces new dependencies and risks; web-based UI may not achieve pixel-perfect Qt parity.

4.3.3 Option 3: Win32 API + GDI/GDI+

Win32 API is the native Windows application programming interface, providing direct access to operating system GUI primitives without framework abstraction layers.

Advantages:

- Zero licensing cost (part of Windows SDK)
- Zero external dependencies (built into Windows OS)
- Smallest possible binary size (<15MB achievable)
- Lowest memory footprint (30-40MB)
- Direct control over every UI element for pixel-perfect replication
- Fastest application startup time
- No compatibility concerns across Windows versions

- Qt itself is built on top of Win32, proving its sufficiency

Disadvantages:

- Windows-only (not cross-platform) — irrelevant since MUP is Windows-only
- More verbose code compared to high-level frameworks
- Manual memory management and resource cleanup
- Steeper learning curve for developers unfamiliar with Win32
- Limited built-in widgets compared to Qt (but sufficient for MUP)

Verdict: Best fit for requirements; maximizes cost savings and performance gains while minimizing risks.

4.4 Framework Comparison Matrix

Table 7: Comprehensive Framework Comparison

Criterion	Qt	Tauri	Win32 API
Annual License Cost	4.25L/dev	0	0
Executable Size	30MB+	15-20MB	<15MB
RAM Usage	80MB	50-60MB	30-40MB
Dependencies	40MB DLLs	100MB WebView2	0MB
Windows Compatibility	Win7-11	Win10+	Win7-11
Development Time	Existing	6-8 weeks	4-6 weeks
OEM Re-validation	No	Possible	No
Long-term Maintenance	Medium	Low-Medium	High

4.5 Recommended Solution Architecture

Based on the evaluation, **Win32 API + GDI+** was selected as the optimal framework.

The proposed architecture consists of:

4.5.1 Backend Layer (Unchanged)

- msecli.exe for firmware operations
- mse-api.dll for device communication
- Firmware binary blob
- XML parser for mup.xml configuration

4.5.2 Platform Abstraction Layer

- OS API wrappers for file I/O, process spawning, threading
- Error handling and logging infrastructure
- Resource management (files, handles, memory)

4.5.3 GUI Layer (New - Win32/GDI+)

- Window creation and event loop management
- Custom-drawn buttons, progress bars, labels
- DPI-aware layout calculations
- Message handling for user interactions

4.5.4 Automation Layer (New)

- Jenkins pipeline integration
- Jira API for metadata extraction
- Automated build scripts
- Validation test framework

4.6 Migration Strategy

The migration was planned in phases to minimize risk:

1. **Phase 1:** Backend refactoring to isolate GUI dependencies
2. **Phase 2:** Win32 window skeleton with basic dialogs
3. **Phase 3:** Custom GDI+ rendering for visual parity
4. **Phase 4:** Integration with backend logic
5. **Phase 5:** Automation pipeline development
6. **Phase 6:** Validation framework creation
7. **Phase 7:** OEM testing and production deployment

This phased approach enabled incremental validation and rollback capability if issues emerged.

4.7 Risk Assessment and Mitigation

Table 8: Risk Mitigation Matrix

Risk	Probability	Mitigation
Visual differences cause OEM rejection	Medium	Pixel-perfect replication; side-by-side comparison testing
Win32 learning curve delays project	Low	Dedicated training; phased implementation
Backend integration issues	Low	Interface contracts; extensive unit testing
DPI scaling problems	Medium	Dynamic coordinate calculations; multi-DPI validation
Performance regressions	Low	Benchmarking at each phase

4.8 Expected Outcomes

The modernization was projected to achieve:

- 100% elimination of Qt licensing costs (21.25L/year for team)
- 50-60% reduction in executable size
- 50-60% reduction in RAM usage
- 90% reduction in build time through automation
- 100% functional equivalence with Qt version
- Enhanced maintainability through cleaner architecture

The following chapters detail the architecture design and implementation of this recommended solution.

5 Future Enhancements

While the current modernization effort has achieved significant improvements in cost reduction, performance optimization, and automation, several opportunities for further enhancement have been identified. These enhancements will build upon the existing architecture to provide even greater operational efficiency and integration capabilities.

5.1 Jira MCP Integration with Jenkins for End-to-End Automation

One of the most promising future enhancements involves deep integration between Jira's Model Context Protocol (MCP), Jenkins CI/CD pipelines, and the MUP tool generation workflow to achieve complete end-to-end automation.

5.1.1 Current Workflow Limitations

The current automated workflow addresses most manual steps but still requires some human intervention:

- Manual creation of Jira tickets for new firmware releases
- Human verification of firmware metadata before build initiation
- Manual triggering of Jenkins pipelines from Jira tickets
- Post-build manual updates to Jira ticket status
- Manual artifact linking and documentation updates

5.1.2 Proposed Integration Architecture

The enhanced workflow would implement bidirectional integration between Jira and Jenkins using webhooks, REST APIs, and MCP connectors:

Step 1: Automated Ticket Creation

- Firmware release detection from upstream systems automatically generates Jira tickets
- Ticket contains structured metadata: drive model, firmware version, supported platforms, validation requirements
- Pre-defined templates ensure consistency and completeness of required fields

Step 2: Jenkins Pipeline Triggering

- Jira webhook monitors ticket transitions (e.g., "Ready for Build" status)
- Webhook payload triggers parameterized Jenkins jobs with ticket metadata
- Jenkins extracts firmware binary URLs, version numbers, and build configurations directly from Jira custom fields

Step 3: Automated Build Execution

- Jenkins pipeline downloads firmware binaries from artifact repositories
- XML configuration files auto-generated from Jira metadata
- Compilation, packaging, and validation tests execute without human intervention
- Build artifacts automatically uploaded to designated storage with proper versioning

Step 4: Bidirectional Status Updates

- Jenkins sends real-time build progress updates to Jira via REST API
- Jira ticket automatically transitions through states: "In Progress" → "Build Complete" → "Validation Pending" → "Released"
- Build logs, test reports, and artifact links automatically attached to Jira tickets
- Failed builds trigger automatic assignment back to development team with failure details

Step 5: Automated Validation Integration

- Successful builds trigger downstream validation jobs

- OEM platform validation results (Dell Dominion) posted back to Jira
- Validation failures create sub-tasks for investigation and resolution
- Full traceability from firmware release to customer delivery

5.1.3 MCP Protocol Benefits

Utilizing Jira's Model Context Protocol provides several advantages:

- **Structured Data Exchange:** MCP provides schema-based data validation ensuring integrity of build parameters
- **Event-Driven Architecture:** Real-time event streaming enables immediate response to workflow state changes
- **Error Handling:** Built-in retry logic and failure recovery mechanisms
- **Security:** Authenticated API calls with token-based authorization
- **Auditability:** Complete audit trail of all automation actions

5.1.4 Implementation Roadmap

The proposed implementation would be executed in phases:

Phase 1 (2-3 weeks): Basic Jira-Jenkins webhook integration for pipeline triggering

Phase 2 (2-3 weeks): Automated metadata extraction and XML generation

Phase 3 (3-4 weeks): Bidirectional status updates and artifact linking

Phase 4 (2-3 weeks): Integration with OEM validation platforms

Phase 5 (1-2 weeks): Dashboard development for visualization and monitoring

5.2 Additional Future Enhancements

Beyond Jira-Jenkins integration, several other enhancements are planned:

5.2.1 Enhanced Security Features

- Code signing certificates for digitally signing executables
- Cryptographic verification of firmware binary integrity before update
- Secure credential management for API authentication
- Encrypted logging for sensitive operational data

5.2.2 Advanced Testing Capabilities

- Performance regression testing framework
- Memory profiling integration in CI/CD pipeline
- Automated accessibility testing for GUI compliance
- Cross-platform compatibility testing (Windows 7-11, Server editions)

5.2.3 Monitoring and Observability

- Telemetry collection from deployed MUP tools in field
- Usage analytics: success rates, failure patterns, common errors
- Integration with enterprise monitoring systems (Splunk, ELK stack)
- Real-time alerting for critical failures in OEM environments

5.2.4 User Experience Improvements

- Multi-language support for international OEM partners
- Customizable branding for different OEM requirements
- Enhanced error messages with troubleshooting guidance
- Recovery mechanisms for interrupted firmware updates

5.3 Expected Impact of Future Enhancements

These planned enhancements are expected to deliver additional benefits:

- **Zero-Touch Workflow:** Complete automation from firmware release to delivery
- **Reduced Time-to-Market:** Firmware release cycles shortened from days to hours
- **Improved Quality:** Automated validation catches regressions earlier
- **Enhanced Visibility:** Real-time dashboards provide stakeholder transparency
- **Scalability:** Infrastructure capable of handling 10x increase in firmware releases

6 Conclusion

The MUP Tool Modernization project successfully addressed critical inefficiencies in Micron Technology’s firmware update tooling infrastructure, achieving substantial cost savings, performance improvements, and operational enhancements while maintaining complete functional compatibility with OEM validation systems.

6.1 Achievement Summary

This project delivered measurable improvements across all key objectives:

Financial Impact: Complete elimination of Qt licensing costs, saving 4.25 lakh per developer annually, or 21.25 lakh per year for a five-developer team. Over a five-year period, this represents cost avoidance exceeding 1 crore.

Performance Optimization: Achieved 57% reduction in executable size (from 30MB+ to 13MB) and 58% reduction in runtime memory usage (from 80MB to 34MB), dramatically improving resource efficiency and deployment footprint.

Operational Efficiency: Fully automated build pipeline reduced manual effort from 35 minutes per firmware release to near-zero, eliminating human error and enabling scalability to increased release cadence.

Quality Assurance: Developed comprehensive 206-test validation framework covering static analysis, functional testing, GUI automation, and end-to-end integration, providing unprecedented confidence in release quality.

Productivity Gains: Delivered 16 story points of value in MS2 sprint, demonstrating that technical debt reduction initiatives yield tangible business benefits.

6.2 Technical Contributions

The project made several significant technical contributions:

Framework Migration Methodology: Demonstrated that mature GUI frameworks like Qt can be successfully replaced with native OS APIs (Win32/GDI+) for domain-specific applications, achieving better performance and eliminating licensing constraints.

Layered Architecture Design: Implemented clean separation of concerns between backend firmware logic, platform abstraction, and UI rendering, improving maintainability and testability.

Automation Framework: Created reusable Jenkins pipeline patterns and Python-based validation frameworks applicable to other internal tools facing similar challenges.

DPI-Aware GUI Engineering: Solved complex DPI scaling challenges through dynamic coordinate calculations, ensuring consistent user experience across diverse display configurations.

6.3 Lessons Learned

Several valuable insights emerged from this project:

1. **Framework Selection is Critical:** Choosing appropriate frameworks based on actual requirements rather than popularity or familiarity can yield dramatic improvements in efficiency and cost.
2. **Technical Debt Has Business Impact:** What appears to be purely technical issues (framework choice, manual processes) directly affect financial outcomes, productivity, and competitiveness.

3. **Automation Pays Dividends:** Investment in comprehensive automation infrastructure delivers compounding returns over time through reduced errors, faster cycles, and improved quality.
4. **Backward Compatibility Enables Adoption:** Maintaining visual and functional parity with existing systems eliminates barriers to adoption and avoids costly re-validation efforts.
5. **Incremental Migration Reduces Risk:** Phased implementation with validation at each stage prevents catastrophic failures and enables course correction.

6.4 Organizational Impact

Beyond technical achievements, this project demonstrated broader organizational benefits:

Cross-Functional Collaboration: Successful coordination between firmware engineers, DevOps teams, QA specialists, and OEM partners validated the importance of stakeholder alignment.

Knowledge Transfer: Comprehensive documentation and training materials ensure long-term maintainability and reduce dependency on individual developers.

Cultural Shift: Raised awareness of technical debt implications and the value of proactive modernization initiatives.

Template for Future Work: Established repeatable patterns and methodologies applicable to other legacy tools in the product portfolio.

6.5 Industry Relevance

This work has relevance beyond Micron Technology:

- Validates native OS APIs as viable alternatives to commercial frameworks for enterprise tools
- Provides cost-benefit analysis methodology for framework migration decisions

- Demonstrates successful integration of modern DevOps practices with legacy systems
- Offers GUI automation patterns using coordinate-based interaction for non-standard controls

6.6 Final Remarks

The MUP Tool Modernization project exemplifies how thoughtful application of software engineering principles, combined with business acumen and operational awareness, can transform legacy systems into efficient, maintainable, and cost-effective solutions.

The project's success validates the hypothesis that enterprise software can achieve dramatic improvements through careful re-engineering without sacrificing functionality or user experience. The architectural patterns, automation frameworks, and validation methodologies developed during this initiative provide a strong foundation for future enhancements and serve as a model for similar modernization efforts across the organization.

Most importantly, this project demonstrates that technical debt reduction is not merely a maintenance activity but a strategic investment that yields measurable returns in cost savings, productivity improvements, and competitive advantage.

Glossary

API (Application Programming Interface): A set of functions, protocols, and tools that enables software components to communicate with each other. In this project, refers to Win32 API for GUI operations and MSE API for SSD communication.

BITS Pilani: Birla Institute of Technology and Science, Pilani - a premier technical institution in India where the author is pursuing undergraduate studies.

CI/CD (Continuous Integration/Continuous Deployment): Software engineering practices that automate building, testing, and deploying applications, implemented in this project using Jenkins.

CLI (Command-Line Interface): A text-based interface for interacting with software,

used for the unattended mode of the MUP tool.

DLL (Dynamic Link Library): A Windows executable file containing code and resources that can be used by multiple programs simultaneously, such as `mse-api.dll` in the MUP architecture.

Dominion: Dell's automated firmware validation platform used to test and certify MUP tool releases before deployment to enterprise customers.

DPI (Dots Per Inch): A measurement of screen resolution and scaling. High-DPI displays require careful coordinate calculations to maintain proper UI layout.

EULA (End User License Agreement): Legal agreement displayed to users before software installation, implemented as the first screen in the MUP tool GUI.

FFI (Foreign Function Interface): A mechanism allowing programs written in one language to call functions written in another, relevant to Rust-C++ integration discussions.

Firmware: Low-level software embedded in hardware devices (SSDs) that controls device operations. The MUP tool's primary purpose is updating SSD firmware.

GDI/GDI+ (Graphics Device Interface): Windows API for 2D graphics rendering, used in this project to draw custom UI elements like buttons and progress bars.

GUI (Graphical User Interface): Visual interface allowing users to interact with software through windows, buttons, and dialogs, replacing the Qt GUI with Win32-based implementation.

Jenkins: Open-source automation server used to implement CI/CD pipelines for automated building, testing, and deployment of the MUP tool.

Jira: Project management and issue tracking software used to coordinate firmware releases and MUP tool generation requests.

MCP (Model Context Protocol): Communication protocol enabling structured data exchange between systems like Jira and Jenkins for workflow automation.

MSE (Micron Storage Executive): Micron's comprehensive SSD management software suite, portions of which are integrated into the MUP tool.

MSVC (Microsoft Visual C++): Microsoft's C++ compiler and development toolchain

used to build the Win32-based MUP tool.

MUP (Manageable Update Package): Micron’s firmware update tool that packages firmware binaries with a user interface into standalone executables for OEM deployment.

OEM (Original Equipment Manufacturer): Companies like Dell, HP, and Lenovo that integrate Micron SSDs into their server products and use MUP tools for field firmware updates.

Practice School II: An industry internship program at BITS Pilani where students work on substantial projects at corporate research stations for six months.

pywinauto: Python library for automating Windows GUI applications, used to implement automated validation tests for the MUP tool interface.

Qt: Cross-platform application framework providing comprehensive GUI development capabilities, used in the original MUP implementation but replaced in this modernization.

RAM (Random Access Memory): Computer memory used for active program execution, where reduced usage was a key optimization goal.

ROI (Return on Investment): Financial metric comparing benefits gained to costs incurred, demonstrating value of the modernization effort.

SDK (Software Development Kit): Collection of tools, libraries, and documentation for developing software, such as the Windows SDK providing Win32 API access.

SSD (Solid State Drive): Storage device using flash memory, manufactured by Micron for enterprise applications requiring periodic firmware updates.

Story Point: Agile project management unit representing complexity and effort of tasks, used to measure productivity improvements achieved.

Tauri: Modern framework for building desktop applications using web technologies and Rust, evaluated but not selected for this project.

Technical Debt: Accumulated cost of suboptimal technical decisions made in the past, such as using Qt for a simple UI, requiring eventual refactoring or modernization.

UI (User Interface): The means by which users interact with software, encompassing both GUI (graphical) and CLI (command-line) interfaces.

WebView2: Microsoft's embedded web browser component for desktop applications, required by Tauri but adding significant dependency overhead.

Win32 API: Microsoft's core application programming interface for Windows operating systems, providing native access to system functions without framework abstraction.

XML (eXtensible Markup Language): Text-based format for storing structured data, used in `mup.xml` to configure firmware metadata and compatibility information.